

6 EE PROJECTS RECRUITERS ACTUALLY WANT TO SEE

Not traffic light simulators. Not LED blinkers.

Projects that map to real job descriptions at real companies.

Full build guides, wiring diagrams, component lists, and
interview prep questions for each project.

By @1nska

NYU Shanghai — 2026

Your Portfolio Is Probably Missing These

Here's what most EE student portfolios look like: a traffic light simulator, a calculator, an LED blinker, maybe a line-following robot from a class project. Recruiters have seen these 10,000 times. They don't differentiate you.

These 6 projects are specifically chosen because they map to skills listed on real job postings at companies like Texas Instruments, Analog Devices, Tesla, Qualcomm, SpaceX, and every hardware startup. Each project proves a different set of professional EE skills — and each one gives you something concrete to discuss in an interview.

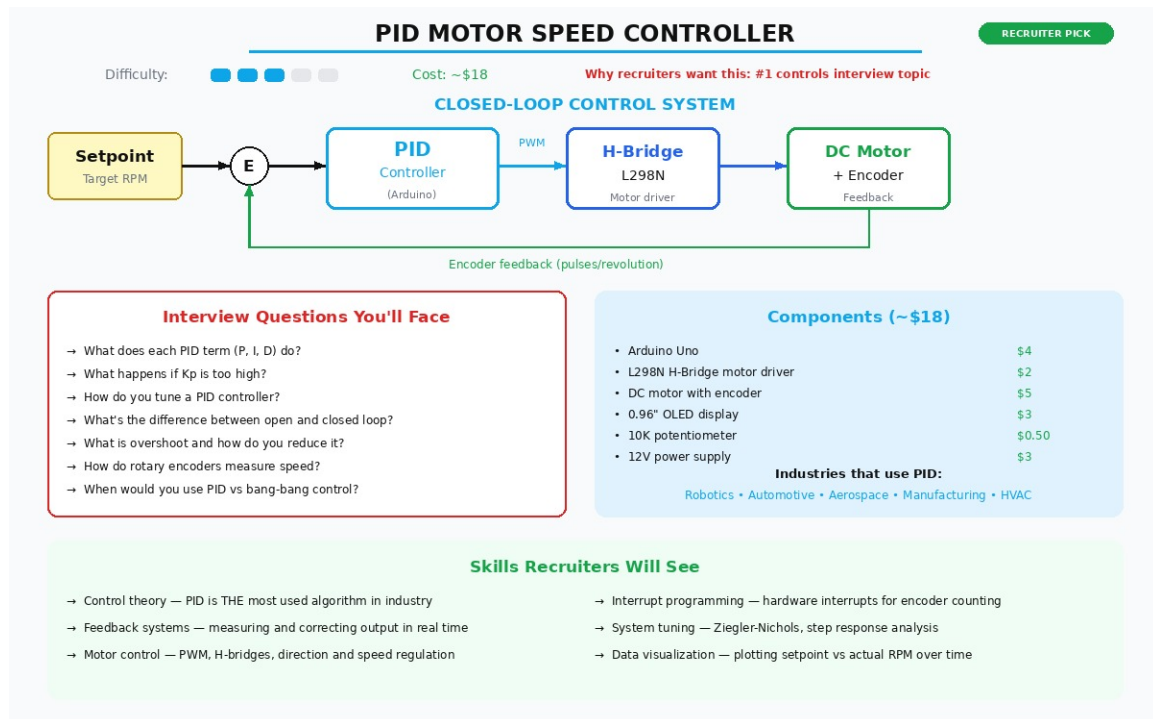
How this guide works: Each project includes a full architecture diagram, component list with costs, step-by-step build instructions, and — most importantly — a list of interview questions you should be able to answer after building it. If you can answer those questions confidently, you're ready for that topic in an interview.

#1

PID Motor Speed Controller

CONTROLS • FEEDBACK • MOTORS • TUNING

Difficulty: ★★★ Beginner-Intermediate | Cost: \$15-18 | Time: 1-2 weekends



Why recruiters want to see this: PID control is the single most asked-about concept in controls engineering interviews. It's used in robotics, automotive, aerospace, manufacturing, HVAC — essentially every system that needs to maintain a target value. If you can explain P, I, and D terms, show a step response plot, and discuss tuning methods, you've demonstrated the core skill of controls engineering.

Build a system that maintains a DC motor at a precise speed using PID control. A rotary encoder measures actual RPM, the Arduino computes the PID output, and an H-bridge adjusts motor power. Display setpoint vs actual RPM on an OLED. Use the Arduino Serial Plotter to visualize your control loop performance — this plot IS your portfolio data.

What you need:

- Arduino Uno — \$4
- L298N H-Bridge motor driver — \$2
- DC motor with rotary encoder — \$5
- 0.96" OLED display (SSD1306) — \$3

- 10K potentiometer (setpoint dial) — \$0.50
- 12V power supply or battery pack — \$3
- Breadboard + jumper wires

Skills this proves:

- **PID control theory** — proportional, integral, derivative terms and what each does
- **Feedback systems** — measuring output and correcting in real time
- **H-bridge motor driving** — PWM speed control, direction switching, braking
- **Rotary encoders** — counting pulses, calculating RPM, interrupt-driven measurement
- **System tuning** — Ziegler-Nichols method, understanding overshoot and settling time
- **Data visualization** — plotting setpoint vs actual for tuning analysis

Step-by-step build:

Step 1: Wire the motor and encoder. Connect the motor to L298N output terminals. Connect encoder output to Arduino D2 (interrupt-capable pin). Use `attachInterrupt()` to count pulses. Calculate RPM: $(\text{pulses} / \text{pulsesPerRev}) \times (60000 / \text{timeInterval})$. Print RPM to Serial Monitor to verify.

Step 2: Implement basic open-loop control. Map a potentiometer on A0 to `analogWrite()` on the L298N enable pin. Verify the motor speed changes smoothly with the knob. Display RPM on the OLED. This is your baseline — open loop, no feedback.

Step 3: Write the PID algorithm from scratch. $\text{Error} = \text{setpoint} - \text{actualRPM}$. $\text{P_term} = K_p \times \text{error}$. $\text{I_term} += K_i \times \text{error} \times \text{dt}$. $\text{D_term} = K_d \times (\text{error} - \text{lastError}) / \text{dt}$. $\text{Output} = \text{P} + \text{I} + \text{D}$. Constrain to 0-255. Don't use a PID library — write it yourself so you can explain every line.

Step 4: Tune using Ziegler-Nichols. Set $K_i=0$, $K_d=0$. Increase K_p until the system oscillates steadily — note this as K_u and the oscillation period as T_u . Set $K_p=0.6 \times K_u$, $K_i=2 \times K_p / T_u$, $K_d=K_p \times T_u / 8$. Fine-tune from there.

Step 5: Add Serial Plotter output. Print setpoint and actual RPM at 50Hz. Open Arduino Serial Plotter — you'll see both lines. Change the setpoint suddenly and watch the step response. Screenshot this — it shows overshoot, settling time, and steady-state error.

Step 6: Document everything. Your GitHub README should include: the PID block diagram, your tuning process, step response screenshots showing before/after tuning, and a table of your final $K_p/K_i/K_d$ values. This is the data that proves you understand controls.

Interview questions you should be ready for:

- What does each PID term (P, I, D) do? What happens if you remove one?
- What happens if K_p is too high? Too low?
- How do you tune a PID controller? Explain Ziegler-Nichols.
- What's the difference between open-loop and closed-loop control?
- What is overshoot? Settling time? Steady-state error?
- How do rotary encoders work? Why use interrupts?
- When would you use PID vs bang-bang (on/off) control?
- How would you add position control (not just speed)?

Tutorials:

- [Video: PID control explained visually](#)
- [Video: Arduino PID motor control tutorial](#)
- [Written: PID theory — National Instruments](#)

Documentation:

- [L298N motor driver guide](#)
- [Arduino interrupt reference](#)
- [Ziegler-Nichols tuning method](#)

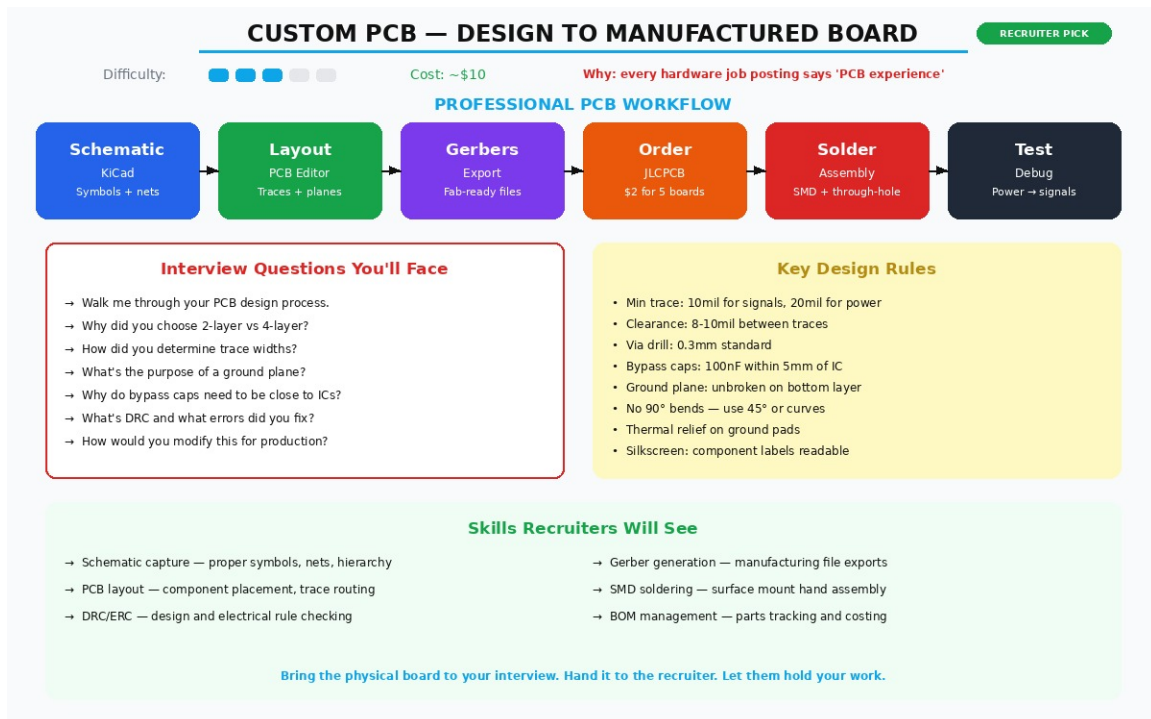
Level up: Add position control — use the encoder to track total rotation and implement a position PID loop. Command the motor to rotate exactly 90° and stop. This is how robotic joints and CNC machines work. Having both speed AND position control on your resume covers the two most common controls interview scenarios.

#2

Custom PCB — Design to Manufactured Board

PCB DESIGN • KICAD • MANUFACTURING • SMD

Difficulty: ★★★ Beginner-Intermediate | Cost: \$8-12 | Time: 2-3 weekends



Why recruiters want to see this: Every hardware job posting says 'PCB design experience preferred.' Most students graduate without ever designing one. Having a physical board you designed in KiCad, manufactured at JLCPCB, and soldered yourself puts you in a completely different category. Bring it to the interview. Hand it to the recruiter. Let them hold your work.

Design an ESP32-based sensor board in KiCad — full schematic, PCB layout with proper ground planes, Gerber export, and manufacturing at JLCPCB (\$2 for 5 boards). Then solder SMD components and test. The result: a professional-quality board that replaces your breadboard prototype.

What you need:

- ESP32-WROOM-32E module — \$3
- BME280 sensor (temp/humidity/pressure) — \$2
- AMS1117-3.3 voltage regulator (SOT-223) — \$0.15
- CP2102N USB-UART bridge — \$1.50
- USB-C connector (SMD) — \$0.30
- 100nF + 10uF capacitors (0805) — \$0.25

- 10K resistors (0603) — \$0.05
- LEDs (0805) + tactile switches — \$0.20
- PCB fabrication at JLCPCB — \$2 for 5 boards

Skills this proves:

- **Schematic capture** — drawing circuits with proper symbols, nets, and hierarchy in KiCad
- **PCB layout** — component placement, trace routing, ground planes, design rules
- **DRC/ERC** — automated design rule and electrical rule checking
- **Gerber generation** — exporting manufacturing files (copper, mask, drill, silkscreen)
- **BOM management** — tracking components, quantities, costs, and availability
- **SMD soldering** — hand soldering surface mount components (0805 and larger)

Step-by-step build:

Step 1: Install KiCad 8 (free). Create a new project. Import the JLCPCB design rules file so everything you design is within their manufacturing capabilities. Add ESP32 symbol library if not included.

Step 2: Draw the schematic. Power path: USB-C → CP2102N → auto-reset circuit → AMS1117-3.3 → ESP32. Add 100nF bypass caps on every IC power pin. Connect BME280 via I2C with 10K pullups. Add 2 LEDs and 2 buttons. Run ERC — fix all errors.

Step 3: Assign footprints. ESP32-WROOM-32E uses a specific castellated pad footprint. AMS1117 uses SOT-223. Caps and resistors use 0805. Double-check EVERY footprint in 3D viewer — a wrong footprint means the part won't fit. This is the #1 first-PCB mistake.

Step 4: Layout the PCB. USB-C at board edge, CP2102N nearby, AMS1117 between USB and ESP32, ESP32 centered, BME280 at board edge away from heat. Bypass caps within 5mm of their ICs. Set trace widths: 10mil signals, 20mil power, 40mil+ high current.

Step 5: Add ground plane. Pour copper on the entire bottom layer connected to GND. Use vias to connect top-side GND pads to the plane. Run DRC — 0 errors, 0 unconnected nets. Export Gerbers and drill files. Upload to jlcpcb.com and verify the preview.

Step 6: Assemble and test. When boards arrive, solder lowest-profile components first (resistors/caps), then ICs, then connectors, then ESP32 last. Use flux generously. Check 3.3V rail with multimeter before powering the ESP32. Flash a test sketch via USB.

Interview questions you should be ready for:

- Walk me through your PCB design process from concept to manufactured board.
- Why did you choose a 2-layer stackup? When would you need 4 layers?
- How did you determine your trace widths for power rails?
- What's the purpose of the ground plane? How does it reduce noise?
- Why do bypass capacitors need to be close to IC power pins?
- What's the difference between HASL and ENIG surface finish?
- What changes would you make for a production run of 1000 units?
- How do you verify that your footprints match the actual components?

Tutorials:

- [Video: KiCad 8 complete PCB tutorial](#)
- [Video: SMD soldering for beginners](#)
- [Written: JLCPCB capabilities and design guide](#)

Documentation:

- [KiCad official getting started guide](#)
- [ESP32-WROOM-32E datasheet](#)
- [AMS1117 datasheet](#)
- [Trace width calculator](#)

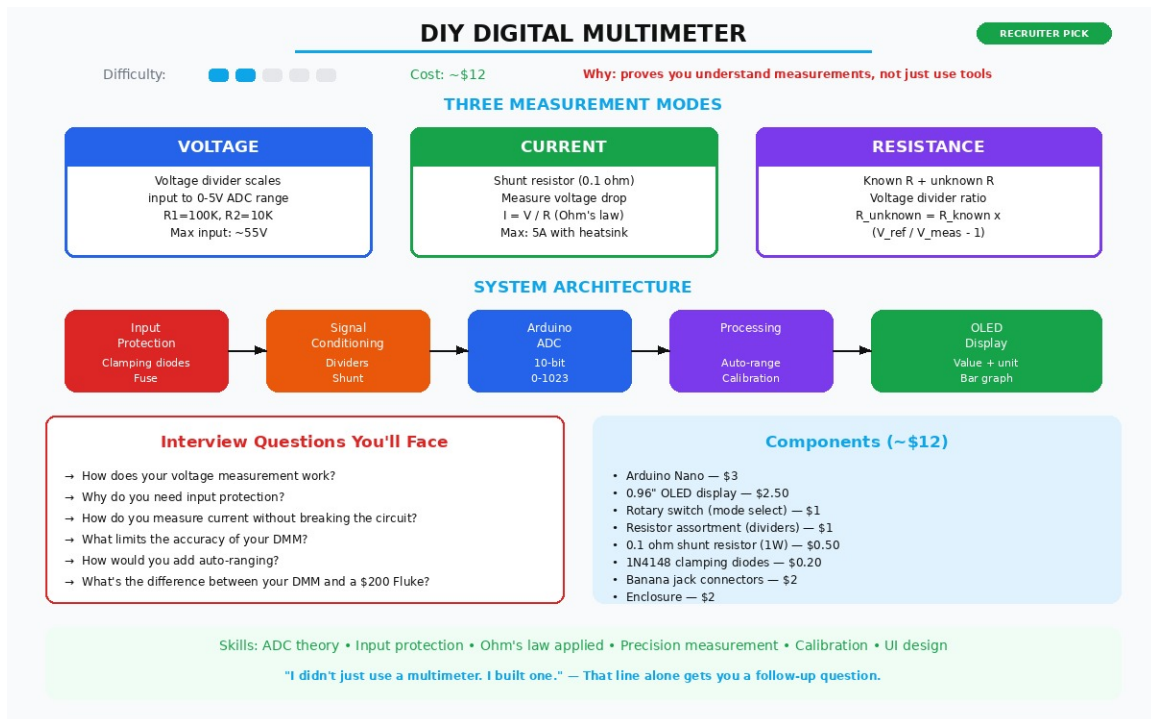
Level up: Get your board assembled by JLCPCB's SMT assembly service — upload your BOM and pick-and-place file. Total cost: ~\$10-15 for 5 fully assembled boards. Now you've experienced the complete professional workflow that hardware startups use.

#3

DIY Digital Multimeter

ADC • MEASUREMENT • INPUT PROTECTION • UI

Difficulty: ★★ ■ ■ ■ Beginner | Cost: \$10-12 | Time: 1-2 weekends



Why recruiters want to see this: Building the tool — not just using it — proves you understand measurements at a fundamental level. When you say 'I built a multimeter that measures voltage, current, and resistance,' interviewers immediately know you understand ADC theory, input protection, Ohm's law applied in practice, and precision measurement.

Build a digital multimeter that measures DC voltage (0-50V), current (0-5A), and resistance (100-100K ohm). A rotary switch selects the mode. An OLED displays the reading with units and a bar graph. Input protection prevents the Arduino from frying if you connect something wrong.

What you need:

- Arduino Nano — \$3
- 0.96" OLED display (SSD1306) — \$2.50
- Rotary switch (3-position, for mode select) — \$1
- Resistor assortment (for voltage dividers + reference) — \$1
- 0.1 ohm 1W shunt resistor (current sensing) — \$0.50
- 1N4148 clamping diodes (input protection) — \$0.20

- Banana jack connectors (red + black) — \$2
- Enclosure — \$2

Skills this proves:

- **ADC theory** — resolution, reference voltage, conversion accuracy and noise
- **Input protection** — clamping diodes, fuses, and current limiting
- **Ohm's law in practice** — voltage dividers for voltage, shunt resistors for current
- **Precision measurement** — calibration, error sources, averaging for noise reduction
- **Auto-ranging concepts** — switching measurement ranges for optimal resolution
- **UI design** — mode selection, display layout, bar graphs, and user feedback

Step-by-step build:

Step 1: Build the voltage measurement circuit. Voltage divider: 100K + 10K gives 11:1 ratio. Max input = 55V → 5V at ADC. Add clamping diodes to 5V and GND rails. Add a 1K series resistor for current limiting. This triple protection saves your Arduino from mistakes.

Step 2: Build the current measurement circuit. A 0.1 ohm shunt resistor in series with the load. Measure the voltage across it: $V = I \times R$, so 1A produces 0.1V. The ADC reads this voltage and calculates current. For better resolution, use a higher shunt value (but it wastes more power).

Step 3: Build the resistance measurement circuit. Known 10K resistor in series with the unknown resistance. Apply 5V. Measure the voltage at the junction. $R_{\text{unknown}} = R_{\text{known}} \times (V_{\text{ref}} / V_{\text{measured}} - 1)$. Multiple reference resistors give different ranges.

Step 4: Wire the rotary switch for mode selection. Position 1 = voltage, Position 2 = current, Position 3 = resistance. The Arduino reads which position is active and switches measurement mode and display accordingly.

Step 5: Program the display. Show the measurement value in large text, the unit (V/A/ohm) beside it, and a bar graph showing the reading relative to the range max. Add an auto-hold feature that freezes the display when the reading stabilizes.

Step 6: Calibrate against a known reference. Compare your readings to a real multimeter. Add calibration constants in code to correct for resistor tolerance and ADC offset. Document your accuracy: $\pm X\%$ for each mode. This data goes in your portfolio.

Interview questions you should be ready for:

- How does your voltage measurement circuit work?
- Why do you need input protection? What happens without it?
- How do you measure current without breaking the circuit?
- What limits the accuracy of your multimeter?
- What's the resolution of a 10-bit ADC at 5V reference?
- How would you add auto-ranging?
- What's the difference between your DMM and a \$200 Fluke?

Tutorials:

- [Video: DIY Arduino multimeter](#)
- [Video: How multimeters work internally](#)
- [Written: ADC fundamentals](#)

Documentation:

- [Arduino ADC reference](#)
- [SSD1306 OLED library](#)
- [Input protection design guide](#)

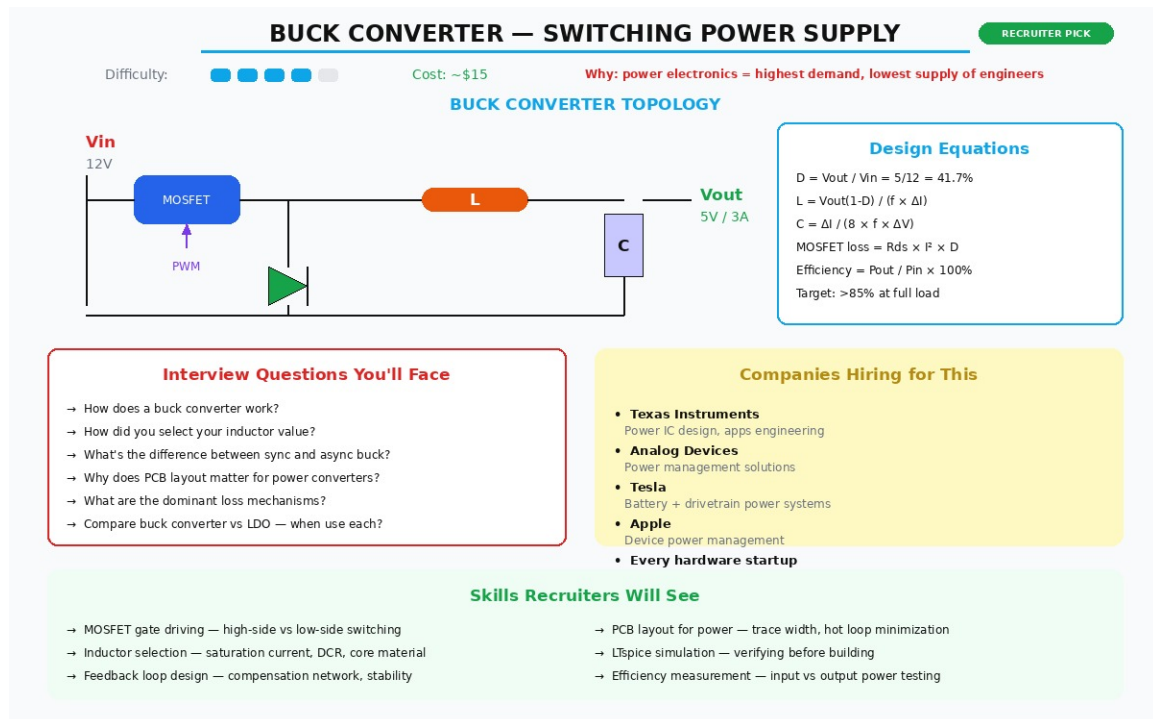
Level up: Add AC voltage measurement with a rectifier circuit and true RMS calculation. Then add a frequency counter mode using timer/counter interrupts. Now you have a 5-mode instrument that rivals basic commercial multimeters.

#4

Buck Converter — Switching Power Supply

POWER • SMPS • MOSFETS • MAGNETICS

Difficulty: ★★★★★ Intermediate | Cost: \$12-18 | Time: 2-3 weekends



Why recruiters want to see this: Power electronics is the most underserved skill in EE hiring. Every electronic device has a power supply, yet most graduates can't design one. Companies like Texas Instruments, Analog Devices, Tesla, and Apple need power engineers. If you can design a buck converter, explain your inductor selection, and show efficiency measurements — you qualify for roles most students don't.

Design a synchronous buck converter: 12V input, 5V output, 3A max, targeting >85% efficiency. Work through component calculations, simulate in LTspice, build on a custom PCB (or breadboard for testing), and measure efficiency across load points. The efficiency curve you produce IS your portfolio proof.

What you need:

- N-channel MOSFETs (IRLZ44N) × 2 — \$1
- IR2104 half-bridge gate driver — \$1
- 10uH power inductor (5A saturation) — \$1.50
- 100uF + 22uF output capacitors — \$0.50

- 22uF input capacitor + 100nF bypass — \$0.40
- Feedback resistors (10K + 3.3K) — \$0.05
- Arduino Nano (for PWM generation) — \$3
- Current sense resistor (10mohm) — \$0.30
- Custom PCB from JLCPCB — \$2

Skills this proves:

- **Buck converter topology** — switching regulation, duty cycle determines V_{out}
- **MOSFET gate driving** — bootstrap circuits, dead time, high-side driving
- **Inductor selection** — saturation current, DCR, core material for switching frequency
- **Feedback loop design** — voltage divider sensing, error amplifier, compensation
- **LTspice simulation** — verifying your design before building hardware
- **Efficiency measurement** — input power vs output power across load points
- **PCB layout for power** — trace width for current, minimizing hot loop area

Step-by-step build:

Step 1: Design calculations. Duty cycle: $D = 5/12 = 41.7\%$. Inductor: $L = V_{out} \times (1-D) / (f_{sw} \times \Delta I)$ — target 20-30% ripple. At 300kHz: $L = 5 \times 0.583 / (300k \times 0.9) = 10.8\mu H \rightarrow$ use 10uH. Peak current = $3 + 0.45 = 3.45A \rightarrow$ choose 5A rated inductor.

Step 2: Simulate in LTspice. Build the full circuit with actual MOSFET spice models. Run transient simulation for 10ms. Verify: output settles to $5.0V \pm 50mV$, inductor current is triangular, no voltage spikes above MOSFET rating. Sweep load from 0.5A to 3A.

Step 3: Build the circuit. On a breadboard for testing, or custom PCB for portfolio quality. Critical: input cap must be within 10mm of high-side MOSFET. Minimize the hot loop area. Gate driver should be close to MOSFETs with short traces.

Step 4: Bring up carefully. Before connecting 12V: verify no shorts between V_{in} and GND. Use a current-limited supply at 100mA first. Check for ~5V output with no load. Gradually increase load: 0.5A $\rightarrow$ 1A $\rightarrow$ 2A $\rightarrow$ 3A.

Step 5: Measure efficiency. At each load point: measure $V_{in} \times I_{in}$ (input power) and $V_{out} \times I_{out}$ (output power). Efficiency = $P_{out} / P_{in} \times 100\%$. Plot efficiency vs load current. Expected: 75% at 0.5A, 85-90% at 2-3A.

Step 6: Document the results. Your README should include: schematic, PCB layout screenshot, efficiency curve plot, output ripple oscilloscope screenshot, and thermal photos if available. This data set is what separates a student project from engineering work.

Interview questions you should be ready for:

- How does a buck converter work? What determines the output voltage?
- How did you select your inductor value? What happens if it saturates?
- What's the difference between synchronous and asynchronous buck?
- Why does PCB layout matter for power converters? What's the 'hot loop'?
- What are the dominant loss mechanisms? How did you optimize efficiency?
- Compare a buck converter to an LDO — when would you use each?
- How would you modify this design for 90%+ efficiency?
- What's the role of the compensation network?

Tutorials:

- [Video: Buck converter design tutorial](#)
- [Video: LTspice power supply simulation](#)
- [Written: TI Power Supply Design Guide](#)

Documentation:

- [IRLZ44N MOSFET datasheet](#)
- [IR2104 gate driver datasheet](#)
- [LTspice download](#)
- [TI WEBENCH Power Designer](#)

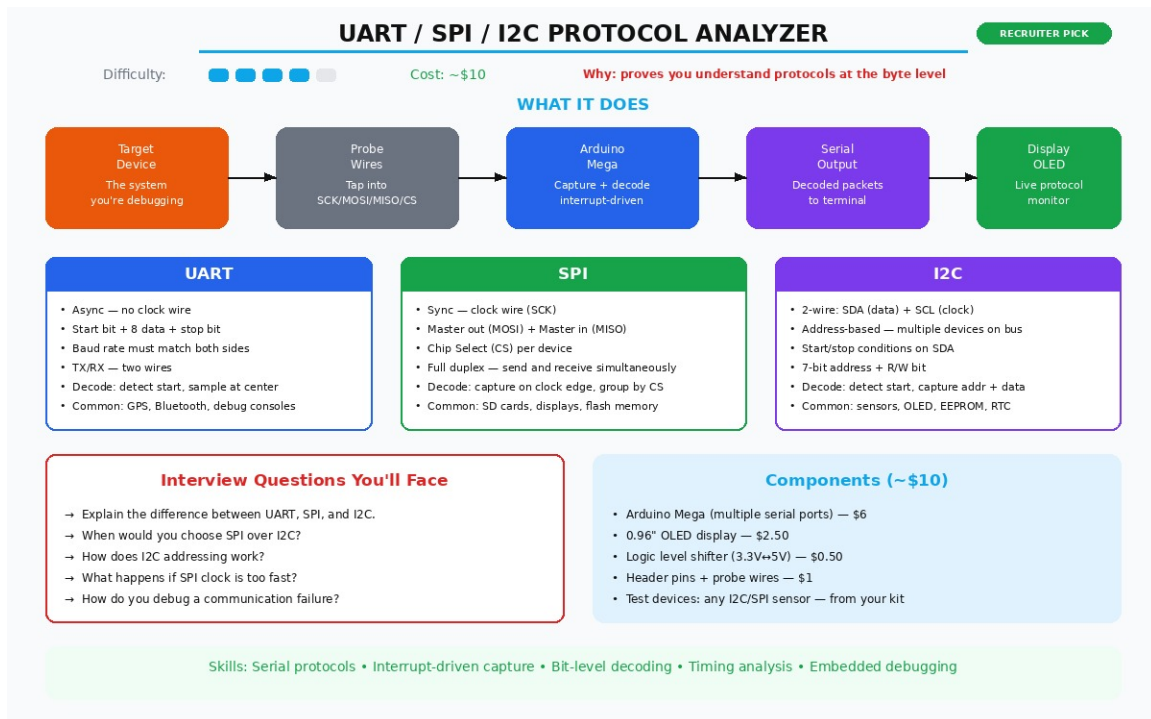
Level up: Design a boost converter (5V → 12V) using the same principles. Having both buck AND boost on your portfolio shows versatility. Then design a buck-boost for battery applications where input voltage can be above or below the output.

#5

UART / SPI / I2C Protocol Analyzer

PROTOCOLS • DEBUGGING • EMBEDDED • SERIAL

Difficulty: ★★★★★ Intermediate | Cost: \$8-12 | Time: 2-3 weekends



Why recruiters want to see this: Every embedded systems job involves debugging communication between chips. Building a protocol analyzer proves you understand UART, SPI, and I2C at the byte level — not just calling library functions, but understanding what's actually happening on the wires. This is a tool that working engineers use daily, and you built it from scratch.

Build a device that passively taps into serial communication lines and decodes the data in real time. Connect to any UART, SPI, or I2C bus and see the decoded bytes, addresses, and timing on your OLED screen or serial terminal. It's a \$200 Saleae logic analyzer — built for \$10.

What you need:

- Arduino Mega (multiple serial ports + interrupts) — \$6
- 0.96" OLED display (SSD1306) — \$2.50
- Bidirectional logic level shifter (3.3V ↔ 5V) — \$0.50
- Header pins + probe wires with clips — \$1
- Test target: any I2C/SPI sensor from your parts bin

Skills this proves:

- **UART protocol** — asynchronous serial, baud rate, start/stop bits, framing
- **SPI protocol** — clock polarity/phase, MOSI/MISO, chip select, full duplex
- **I2C protocol** — addressing, start/stop conditions, ACK/NACK, bus arbitration
- **Interrupt-driven capture** — using hardware interrupts for precise timing
- **Bit-level decoding** — converting raw signal edges to meaningful bytes
- **Embedded debugging** — the most practical skill for any firmware role

Step-by-step build:

Step 1: Build the UART decoder first (simplest). Connect a probe wire to a UART TX line. Use Serial1 on the Mega to capture incoming bytes. Auto-detect baud rate by measuring the width of the start bit. Display decoded ASCII and hex on the OLED. Test by decoding a GPS module's NMEA output.

Step 2: Build the SPI decoder. Connect probes to SCK, MOSI, MISO, and CS lines. Use pin change interrupts on SCK to capture data bits on each clock edge. Group bits into bytes. When CS goes high, that's the end of a transaction. Display: [CS] [command byte] [data bytes] for each transaction.

Step 3: Build the I2C decoder. Connect probes to SDA and SCL. Detect start condition (SDA falls while SCL is high). Capture 8 bits + ACK on each clock cycle. First byte = address + R/W bit. Display: [START] [addr] [R/W] [data] [ACK/NACK] [STOP] for each transaction.

Step 4: Add protocol auto-detection. If you see a clock line with regular edges + CS pin → SPI. If you see two wires with start/stop conditions → I2C. If you see a single async line → UART. Select mode automatically based on what you connect to.

Step 5: Add timing analysis. Measure and display: clock frequency (SPI/I2C), baud rate (UART), time between transactions, and total transaction duration. This timing data is often the key to finding bugs in real embedded systems.

Step 6: Build a decode log. Store the last 50 decoded transactions in a circular buffer. Scroll through them on the OLED. Optionally stream the full decode log over USB to a Python script that saves it to a file for later analysis.

Interview questions you should be ready for:

- Explain the difference between UART, SPI, and I2C.
- When would you choose SPI over I2C? What are the tradeoffs?
- How does I2C addressing work? What happens with address conflicts?
- What determines the maximum SPI clock speed?
- How do you debug a communication failure between two chips?
- What's a logic level shifter and when do you need one?
- How would you decode a protocol you've never seen before?

Tutorials:

- Video: [UART vs SPI vs I2C explained](#)
- Video: [Building a logic analyzer with Arduino](#)
- Written: [SparkFun — SPI tutorial](#)

Documentation:

- [I2C specification \(NXP\)](#)
- [SPI protocol reference](#)
- [Arduino Mega pinout \(interrupt pins\)](#)
- [UART protocol reference](#)

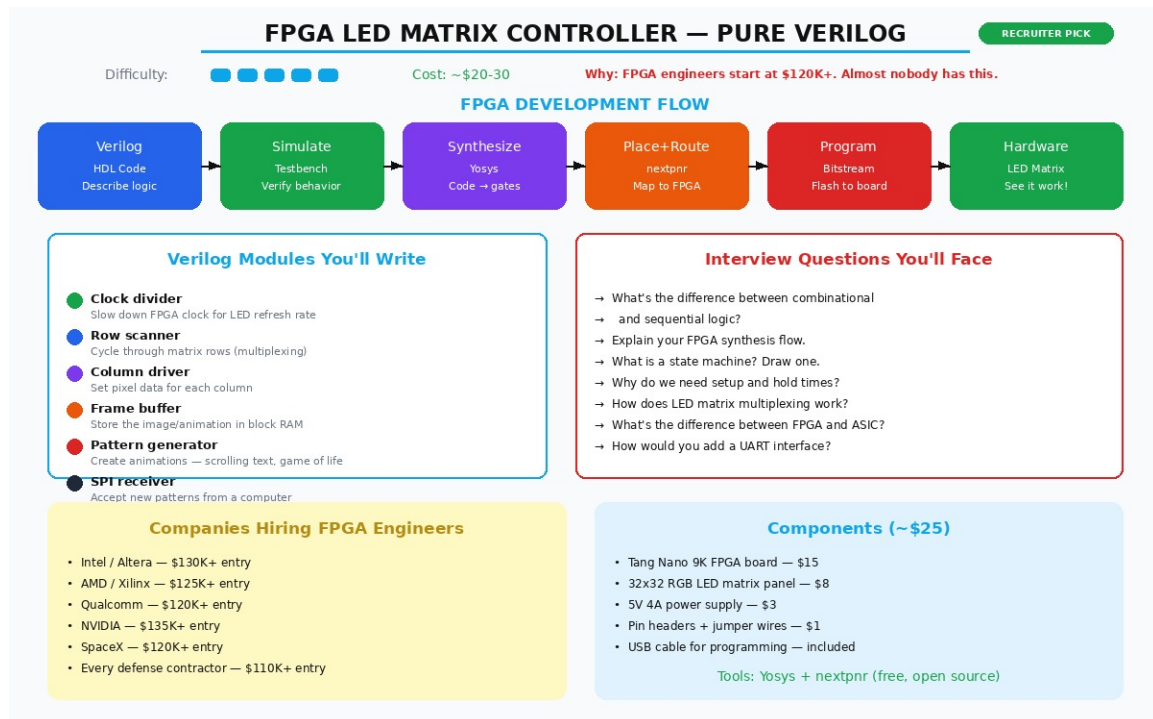
Level up: Add a trigger system — start capturing only when you see a specific address or byte pattern. This is how professional protocol analyzers work (Saleae, Teledyne). Then add a Python GUI that displays decoded packets with color-coded fields and search/filter capability.

#6

FPGA LED Matrix Controller — Pure Verilog

FPGA • VERILOG • DIGITAL DESIGN • HDL

Difficulty: ★★★★★ Intermediate-Advanced | Cost: \$20-30 | Time: 3-4 weekends



Why recruiters want to see this: FPGA engineers are among the highest-paid in EE — \$120-180K entry level at Intel, AMD, Qualcomm, NVIDIA, SpaceX. The supply of qualified engineers is tiny because most universities barely cover FPGAs. If you can show a working FPGA project written in Verilog, you immediately qualify for roles most graduates can't even apply for. This single project can change your career trajectory.

Write a Verilog controller that drives a 32×32 RGB LED matrix panel. No Arduino. No library. Pure digital logic — you design the clock divider, row scanner, column driver, frame buffer, and pattern generator as separate Verilog modules. Simulate each module with testbenches, synthesize with open-source tools, and program onto a \$15 FPGA board.

What you need:

- Tang Nano 9K FPGA dev board (Gowin GW1NR-9) — \$15
- 32×32 RGB LED matrix panel (HUB75 interface) — \$8
- 5V 4A power supply (for LED panel) — \$3
- Pin headers + jumper wires — \$1

- USB-C cable (for programming) — included

Skills this proves:

- **Verilog HDL** — writing synthesizable hardware descriptions
- **Digital design** — combinational logic, sequential logic, state machines
- **Simulation** — writing testbenches, verifying in Icarus Verilog + GTKWave
- **Synthesis** — understanding how Verilog becomes gates and flip-flops
- **LED matrix multiplexing** — row scanning, timing, brightness control
- **Memory design** — using block RAM for frame buffers

Step-by-step build:

Step 1: Set up the toolchain. Install Yosys (synthesis), nextpnr-gowin (place and route), and openFPGALoader (programming). All free and open source. Alternatively use Gowin's IDE. Write a simple LED blinker in Verilog to verify your toolchain works.

Step 2: Write the clock divider module. The Tang Nano 9K has a 27MHz crystal. You need a slower clock for LED refresh. Implement a configurable divider: $\text{input clock} / 2^N = \text{output clock}$. Write a testbench that verifies the output frequency. Simulate in Icarus Verilog.

Step 3: Write the row scanner. The 32×32 matrix has 16 row pairs (dual scan). A 4-bit counter selects which row pair is active. The scanner cycles through all 16 rows fast enough that persistence of vision makes it look like all rows are on simultaneously. Target: >100Hz refresh rate.

Step 4: Write the column driver. For each active row, shift out 32 pixels × 3 colors (RGB) = 96 bits of data through the HUB75 interface. This requires: shift clock, data lines (R1/G1/B1 for top half, R2/G2/B2 for bottom), latch, and output enable signals with correct timing.

Step 5: Add the frame buffer. Store a 32×32 RGB image in block RAM. The column driver reads pixel data from the buffer for the current row. Write a pattern generator that fills the buffer with animations: solid colors, scrolling rainbow, game of life, scrolling text.

Step 6: Add external input. Implement a SPI receiver module that accepts new frame data from a computer. A Python script sends images over SPI to update the display. Now you have a computer-controlled LED display running on pure FPGA logic.

Interview questions you should be ready for:

- What's the difference between combinational and sequential logic?
- Explain the FPGA synthesis flow — what happens between code and hardware?
- What is a state machine? Draw one for your row scanner.
- How does LED matrix multiplexing work? What determines refresh rate?
- Why do we need setup and hold times? What happens when violated?
- What's the difference between an FPGA and an ASIC?
- How would you add PWM brightness control per pixel?
- What tools did you use for simulation and synthesis?

Tutorials:

- [Video: Verilog tutorial for beginners](#)
- [Video: FPGA LED matrix project](#)
- [Written: FPGA 101 getting started](#)

Documentation:

- [Tang Nano 9K documentation](#)
- [HUB75 LED matrix protocol](#)
- [Yosys open synthesis suite](#)
- [Icarus Verilog simulator](#)
- [GTKWave waveform viewer](#)

Level up: Implement a simple RISC-V CPU core (RV32I subset) on the same FPGA. Use it to generate patterns on the LED matrix. Now your LED display is driven by a CPU you designed from scratch. That's the kind of project that makes FPGA hiring managers schedule an interview on the spot.

Quick Reference

#	Project	Key Skills	Target Roles	Cost
1	PID Motor Controller	Controls + Feedback	Controls / Robotics	\$18
2	Custom PCB	KiCad + Manufacturing	Hardware Engineer	\$10
3	Digital Multimeter	ADC + Measurement	Test / Analog	\$12
4	Buck Converter	Power Electronics	Power Engineer	\$15
5	Protocol Analyzer	UART/SPI/I2C	Embedded / Firmware	\$10
6	FPGA LED Matrix	Verilog + Digital	FPGA / ASIC	\$25

Priority order if you're short on time: PCB (#2) is the highest-ROI single project — it applies to every hardware role. PID (#1) is second — it covers the most common interview topic. Buck converter (#4) third — it opens the door to power electronics roles. Do those 3 and you're ahead of 90% of EE graduates.

Follow @1nska on Instagram for build logs, career content, and new project guides. DM me your completed builds — I feature the best ones in my stories.

— Enes (@1nska)
NYU Shanghai, 2026